

AI & DATA SCIENCE STUDY SERIES

RAG

Retrieval-Augmented Generation

Teaching AI to Look Things Up Before It Speaks

Target Audience	Difficulty	Modules	Reading Time
Beginners — Intermediate	Beginner-Friendly	9 Core Modules	50-65 Minutes

What You Will Learn:

- Understand the core problem RAG solves and why it exists
- Learn how the Retrieve → Augment → Generate pipeline works step by step
- Understand embeddings, vector databases, and semantic search
- Explore advanced RAG techniques: Hybrid Search, Re-ranking, HyRAG, and more
- Compare RAG vs Fine-tuning — when to use each
- See real-world RAG applications across industries
- Learn the key tools, frameworks, and metrics for building RAG systems

These notes are designed as structured, exam-ready material. Read each module in order for the best learning experience. Key Concept, Note, and Takeaway boxes are ideal for quick revision.

Module 1: The Problem RAG Solves

1.1 The Core Challenge with LLMs

Large Language Models (LLMs) like GPT-4, Claude, and Llama are incredibly powerful. But they have a fundamental limitation — they are frozen in time.

KEY CONCEPT
An LLM is trained on data up to a certain date (its 'knowledge cutoff'). After training, the model's internal knowledge does not update.
If you ask an LLM about events after its training cutoff, or about private company documents it has never seen, it cannot answer accurately — and it may confidently hallucinate a wrong answer.

1.2 Three Core Problems RAG Addresses

Problem	Description	Example Failure
Knowledge Cutoff	Model has no knowledge of events after training date	Asking about news from last week — model invents an answer
Private / Domain Data	Model was not trained on your company's internal documents	Asking about your own HR policy — model guesses or hallucinates
Hallucination	Model confidently states incorrect information when it does not know	Asking a rare legal question — model fabricates a convincing but wrong answer

1.3 What is RAG?

RAG stands for Retrieval-Augmented Generation. It is a technique that gives an LLM access to external knowledge at inference time — meaning the moment you ask it a question.

KEY CONCEPT
RAG = Giving the LLM a 'cheat sheet' of relevant information just before it answers.
Instead of relying only on memorised training data, the model first retrieves relevant documents from a knowledge base, then uses them to generate a grounded, accurate answer.

1.4 A Simple Analogy

Imagine two types of exam students:

Student Type	Behaviour
--------------	-----------

Closed-Book Student (Standard LLM)	Relies entirely on memory. Might get things right for common topics, but can confidently misremember details for niche questions.
Open-Book Student (RAG-powered LLM)	Looks up the relevant section of their textbook before answering. More accurate, grounded, and less likely to fabricate.

KEY TAKEAWAY

RAG does not change the LLM — it changes what the LLM sees before it answers. By injecting retrieved, relevant context into the prompt, RAG dramatically reduces hallucination and enables the model to answer questions about private or recent information.

Module 2: How RAG Works — The Pipeline

RAG follows a clear, three-phase pipeline. Every RAG system — from the simplest to the most sophisticated — follows this core structure.

2.1 The RAG Pipeline Overview

Phase	Name	What Happens
Phase 1	Indexing (Offline)	Documents are split, embedded, and stored in a vector database. Done once, before any queries.
Phase 2	Retrieval (Online)	User's question is embedded and used to search the vector DB for the most relevant document chunks.
Phase 3	Generation (Online)	Retrieved chunks + user question are sent to the LLM, which generates a grounded final answer.

2.2 Phase 1: Indexing — Preparing Your Knowledge Base

This is the offline setup phase. Before a user can ask questions, your documents must be processed and stored.

Step	What Happens
1. Document Loading	Collect all source documents: PDFs, Word files, web pages, databases, etc.
2. Text Splitting / Chunking	Break documents into smaller pieces (called chunks). Typically 200-1000 tokens each.
3. Embedding Generation	Each chunk is converted into a numerical vector (embedding) using an embedding model.
4. Vector Storage	Embeddings are stored in a vector database (Pinecone, Chroma, Weaviate, etc.).

NOTE

Chunking strategy matters enormously. Chunks too small lose context; chunks too large reduce retrieval precision.

Common approaches: fixed-size chunks, sentence-based chunks, semantic chunks (grouped by meaning), and recursive character splitting.

2.3 Phase 2: Retrieval — Finding the Right Information

When a user asks a question, the system must find the most relevant chunks from the vector database.

Step	What Happens
1. Query Embedding	The user's question is converted into a vector using the same embedding model used during indexing.
2. Similarity Search	The system searches the vector DB for chunks whose embeddings are closest to the query embedding.
3. Ranking & Selection	The top-k most similar chunks are selected (commonly k=3 to 10).
4. Context Assembly	Selected chunks are assembled into a context block to be added to the prompt.

KEY CONCEPT

Semantic Similarity: The search is not keyword-based — it is meaning-based. A query about 'car fuel efficiency' will match documents about 'vehicle petrol consumption' even without exact word overlap.
This is the power of embeddings — they capture meaning, not just words.

2.4 Phase 3: Generation — The LLM Answers

The final phase is where the LLM generates the answer using the retrieved context.

Step	What Happens
1. Prompt Construction	A prompt is built containing: system instructions + retrieved chunks + user's original question.
2. LLM Inference	The prompt is sent to the LLM (GPT-4, Claude, Llama, etc.).
3. Grounded Response	The LLM generates an answer, constrained by the provided context rather than hallucinating.
4. (Optional) Citation	The system can include source references so users know where the answer came from.

EXAMPLE

Example RAG Prompt Structure:

System: You are a helpful assistant. Answer the user's question using ONLY the context provided below. If the answer is not in the context, say 'I don't know.'

Context:
[Chunk 1: ...relevant text from document A...]
[Chunk 2: ...relevant text from document B...]

User Question: What is our company's refund policy for digital products?

KEY TAKEAWAY

The RAG pipeline: Index (offline) → Retrieve (at query time) → Generate (with context).
The LLM itself does not change — what changes is the information it receives in its prompt.
Good retrieval is the foundation of good RAG. If the wrong chunks are retrieved, even the best LLM will fail.

Module 3: Embeddings and Vector Databases

Embeddings and vector databases are the technological heart of RAG. Understanding them is essential for building and improving RAG systems.

3.1 What is an Embedding?

An embedding is a numerical representation of text — a list of hundreds or thousands of numbers (a vector) that captures the meaning of that text.

KEY CONCEPT

Words or sentences with similar meanings produce vectors that are close together in mathematical space.
Example: 'Dog' and 'Puppy' will have very similar vectors. 'Dog' and 'Spaceship' will have very different vectors.
An embedding model (like text-embedding-ada-002 or sentence-transformers) does the conversion.

3.2 How Similarity Search Works

Once text is converted to vectors, we can measure how similar two pieces of text are by measuring how close their vectors are. This is called a similarity metric.

Metric	What It Measures	Best For
Cosine Similarity	Angle between two vectors (direction)	Most common in RAG — works well for text
Euclidean Distance	Straight-line distance between vectors	Better for low-dimensional data
Dot Product	Magnitude + direction	Fast, used when vectors are normalised

NOTE

For RAG, Cosine Similarity is almost always the right choice.
Scores range from 0 (completely different) to 1 (identical). Typically, scores above 0.75 indicate good semantic relevance.

3.3 Popular Embedding Models

Model	Provider	Notes
text-embedding-ada-002	OpenAI	Most widely used; 1536 dimensions; great all-round performance

text-embedding-3-large	OpenAI	Newer, better quality; 3072 dimensions; more expensive
embed-english-v3.0	Cohere	Strong multilingual support; good for enterprise use
all-MiniLM-L6-v2	Hugging Face (SBERT)	Open-source, fast, lightweight; great for local deployment
BGE-M3	BAAI (open-source)	State-of-the-art open-source multilingual embedding model
Gemini text-embedding	Google	High quality; integrates well with Google Cloud

3.4 Vector Databases

A vector database is a specialised database designed to store and efficiently search high-dimensional vectors. Standard SQL databases cannot do this well.

Vector DB	Type	Best For
Pinecone	Managed cloud	Production use; fully managed; easy to scale
Chroma	Open-source, local	Local development; great for prototyping RAG apps
Weaviate	Open-source / cloud	Hybrid search (vector + keyword); strong schema support
Qdrant	Open-source / cloud	High performance; good for large-scale production
Milvus	Open-source / cloud	Enterprise-scale; strong community
FAISS	Open-source library	Facebook AI library; runs in-memory; no server needed
pgvector	PostgreSQL extension	Add vector search to an existing PostgreSQL database
Azure AI Search	Managed cloud	Best for Microsoft / Azure ecosystem

TIP

Starting out? Use Chroma for local prototyping — it requires no setup and works entirely in Python.

Moving to production? Pinecone or Qdrant are the most popular managed options.

Already using PostgreSQL? Add pgvector — it avoids introducing a new database system.

Module 4: Advanced RAG Techniques

Basic RAG works well, but real-world applications often need more sophisticated approaches. This module covers the most important advanced techniques.

4.1 The Problem with Naive RAG

Standard (naive) RAG retrieves chunks by semantic similarity alone. This has several known weaknesses:

Weakness	Description
Keyword Mismatch	Semantic search can miss exact keyword matches that are crucial for the answer
Chunk Boundary Issues	The answer might span across two chunks that are retrieved separately
Lost in the Middle	LLMs tend to pay less attention to information in the middle of a long context
Irrelevant Top Results	Top-k chunks may include some that are semantically similar but not truly relevant
No Query Understanding	If the user's query is ambiguous or complex, retrieval quality suffers

4.2 Hybrid Search

Hybrid search combines semantic (vector) search with traditional keyword (BM25) search. This gets the best of both worlds.

Search Type	Strength
Semantic Search (Vector)	Finds conceptually similar content even with different wording
Keyword Search (BM25)	Finds exact keyword and phrase matches; crucial for proper nouns, codes, IDs
Hybrid Search (Combined)	Scores are merged (using Reciprocal Rank Fusion or weighted average) for maximum recall

EXAMPLE

Example: Query = 'What is the interest rate on Loan Product XYZ-2024?'

Semantic search finds documents about interest rates and loan products generally.

Keyword search finds documents containing the exact string 'XYZ-2024'.

Hybrid search returns documents that are both semantically relevant AND contain the exact product code.

4.3 Re-ranking

After retrieving the top-k chunks, a re-ranker model re-scores them for relevance to the query. This adds a powerful second filter.

Stage	Model Type
Stage 1: Retrieval	Fast embedding model — retrieves top 20-50 candidates quickly
Stage 2: Re-ranking	Slower, more accurate cross-encoder model — re-scores and picks the best 3-5

NOTE

Popular re-ranker models: Cohere Rerank, BGE-Reranker, ColBERT, ms-marco-MiniLM-L-12-v2. Re-ranking significantly improves precision at a small cost in latency. Worth the trade-off for most production systems.

4.4 Query Transformation Techniques

Before retrieval, the user's query can be rewritten or expanded to improve retrieval quality.

Technique	How It Works
Query Rewriting	LLM rephrases the query to be clearer and more retrieval-friendly
HyDE (Hypothetical Document Embedding)	LLM generates a hypothetical answer; that answer is embedded and used for retrieval
Multi-Query Retrieval	LLM generates multiple variations of the query; each is used to retrieve chunks; results are merged
Step-Back Prompting	LLM generates a more general version of the query to retrieve broader context first
Sub-Query Decomposition	Complex multi-part questions are broken into simpler sub-queries, each retrieved separately

4.5 Contextual Compression

Instead of sending entire retrieved chunks to the LLM, contextual compression extracts only the most relevant sentences or passages from each chunk.

KEY CONCEPT

Benefit: Reduces noise in the prompt, lowers token cost, and helps the LLM focus on what truly matters.
Implementation: Use an LLM to filter or compress each retrieved chunk before adding it to the final prompt.

4.6 Parent-Child Chunking (Small-to-Big Retrieval)

A sophisticated chunking strategy that maintains two levels of chunks — small for precise retrieval, large for full context.

Chunk Type	Purpose
Child Chunks (small, ~100 tokens)	Used for retrieval — small chunks match queries more precisely
Parent Chunks (large, ~500-1000 tokens)	Used for generation — when a child chunk is retrieved, the full parent chunk is sent to the LLM

4.7 Summary of Advanced RAG Techniques

Technique	Primary Benefit
Hybrid Search	Higher recall — catches both semantic and keyword matches
Re-ranking	Higher precision — removes irrelevant results from top-k
Query Rewriting	Better queries lead to better retrieval
HyDE	Improves retrieval for complex or abstract queries
Multi-Query	Reduces sensitivity to exact query phrasing
Contextual Compression	Cleaner context, lower cost, less distraction for the LLM
Parent-Child Chunking	Precise retrieval + full context for generation
Self-RAG	Agent decides when and whether to retrieve; reflects on output quality

Module 5: RAG vs. Fine-Tuning

A common question: should I use RAG or fine-tune the LLM? The answer depends on your use case. Here is a complete comparison.

5.1 Head-to-Head Comparison

Dimension	RAG	Fine-Tuning
What it does	Injects external knowledge at query time	Bakes knowledge into model weights during training
Knowledge Updates	Easy — update the vector DB anytime	Hard — requires retraining with new data
Cost	Lower upfront; scales with query volume	High upfront training cost; lower inference cost
Accuracy on Private Data	Very good — direct document retrieval	Good — but knowledge may get diluted or forgotten
Hallucination Risk	Lower — answers grounded in retrieved docs	Higher — relies on generalised learned patterns
Setup Complexity	Medium — requires indexing pipeline	High — requires training data, GPU, evaluation
Best For	Dynamic, frequently changing knowledge bases	Changing model style, tone, or learning new skills
Citation / Traceability	Yes — can cite retrieved sources	No — knowledge is implicit in weights
Use in Production	Most enterprise RAG systems today	Less common; used for specialised domain adaptation

5.2 When to Choose RAG

- Your knowledge base changes frequently (news, policies, product catalogues)
- You need traceable, source-cited answers
- You are working with large amounts of private company data
- You want to reduce hallucinations for factual Q&A tasks
- You need to get a system running quickly without large compute budgets

5.3 When to Choose Fine-Tuning

- You want to change how the model writes or communicates (e.g. a specific brand tone)
- You need the model to learn a new structured format or specialised vocabulary
- Your knowledge is static and unlikely to change
- You need very fast inference with minimal prompt size

5.4 The Best of Both: RAG + Fine-Tuning

In many production systems, RAG and fine-tuning are combined:

- Fine-tune: to teach the model domain-specific language, terminology, and response style.
- RAG: to provide up-to-date, specific factual knowledge at query time.

EXAMPLE

Example: A medical AI assistant might be fine-tuned on clinical communication style, then use RAG to retrieve up-to-date drug information, clinical guidelines, and patient records.

Module 6: RAG Evaluation — Measuring Quality

Building a RAG system is only half the job. Measuring its quality rigorously is equally important. Poor evaluation leads to poor systems.

6.1 The Three Pillars of RAG Quality

Pillar	Question It Answers	What to Measure
Retrieval Quality	Did we retrieve the right chunks?	Are the retrieved documents actually relevant to the query?
Generation Quality	Did the LLM answer correctly?	Is the answer faithful to the retrieved context? Is it complete?
End-to-End Quality	Is the final answer good?	Does the answer correctly address the user's original question?

6.2 Key Retrieval Metrics

Metric	What It Measures
Context Precision	Of all retrieved chunks, what proportion were actually relevant? (Precision = no junk)
Context Recall	Of all relevant chunks in the DB, what proportion were actually retrieved? (Recall = nothing missed)
Mean Reciprocal Rank (MRR)	How highly ranked was the first truly relevant result? Higher rank = better.
Hit Rate (Recall@k)	Did at least one of the top-k retrieved chunks contain the answer?

6.3 Key Generation Metrics

Metric	What It Measures
Faithfulness	Is the generated answer supported by the retrieved context? (No hallucinations)
Answer Relevance	Does the answer actually address the user's question?
Answer Correctness	Is the answer factually correct compared to ground truth?
Answer Completeness	Does the answer cover all necessary aspects of the question?

6.4 RAG Evaluation Frameworks

Framework	Description
-----------	-------------

RAGAS	Most popular open-source RAG evaluation library. Automates scoring of Faithfulness, Answer Relevance, Context Precision, Context Recall.
TruLens	Tracks and evaluates LLM app performance over time. Strong for production monitoring.
LangSmith	LangChain's tracing and evaluation platform. Great if you are building with LangChain.
DeepEval	Testing framework for LLM apps. Supports many metrics and CI/CD integration.
Arize Phoenix	Observability platform for LLM and RAG systems. Strong real-time monitoring.

TIP

Start evaluation early — do not wait until the end. Evaluate after each change to the pipeline. Use RAGAS for a quick, automated baseline. It requires no labelled ground truth for some metrics.
Build a 'golden dataset' of 50-100 question-answer pairs from your domain to use as a consistent benchmark.

Module 7: Tools and Frameworks for Building RAG

The RAG ecosystem has matured rapidly. Here is a complete map of the tools you need at each layer of the stack.

7.1 The RAG Tech Stack

Layer	Role	Popular Options
Document Loaders	Ingest documents from various sources	LangChain Document Loaders, LlamaIndex Readers, Unstructured.io
Text Splitters	Chunk documents intelligently	LangChain RecursiveCharacterTextSplitter, SemanticChunker
Embedding Models	Convert text to vectors	OpenAI, Cohere, BAAI/BGE, Sentence Transformers
Vector Databases	Store and search embeddings	Pinecone, Chroma, Weaviate, Qdrant, Milvus, pgvector
Retrieval Logic	Query transformation, hybrid search, re-ranking	LangChain Retrievers, LlamaIndex Query Engines, Cohere Rerank
LLM / Generator	Generate the final answer	GPT-4o, Claude 3.5, Gemini, Llama 3, Mistral
Orchestration	Tie the pipeline together	LangChain, LlamaIndex, Haystack, custom Python
Evaluation	Measure system quality	RAGAS, TruLens, LangSmith, DeepEval
Observability	Monitor production systems	LangSmith, Arize Phoenix, Langfuse

7.2 LangChain vs. LlamaIndex

The two most popular RAG frameworks. Understanding their differences helps you choose the right one.

Aspect	LangChain vs. LlamaIndex
Primary Focus	LangChain: General LLM application building (chains, agents, RAG). LlamaIndex: Data framework specifically optimised for RAG and retrieval.
RAG Complexity	LangChain is flexible but requires more manual configuration. LlamaIndex has more built-in RAG optimisations out of the box.
Advanced RAG	LlamaIndex has stronger built-in support for advanced techniques (HyDE, re-ranking, sub-query, etc.).
Agents	LangChain has a richer agent ecosystem and integrations.

Learning Curve	LangChain: slightly steeper. LlamaIndex: more focused, easier for pure RAG tasks.
Recommendation	Use LlamaIndex for RAG-first projects. Use LangChain if you need agents + RAG together.

7.3 Managed RAG Solutions

For teams that want RAG without building the infrastructure:

Product	Description
Azure AI Search + OpenAI	Microsoft's managed RAG stack; integrates with Azure ecosystem
Amazon Bedrock + Knowledge Bases	AWS managed RAG; supports multiple LLMs on AWS
Google Vertex AI Search	Google's managed enterprise search with generative AI
Vectara	Purpose-built RAG platform; handles the entire pipeline as a service
Perplexity API	RAG-powered search API; handles retrieval from the web

Module 8: Real-World RAG Applications

RAG is one of the most widely deployed AI techniques in production today. Here are the most impactful applications across industries.

8.1 Applications by Industry

Industry	RAG Application	How It Works
Legal	Contract and case law Q&A	Lawyers query a vector DB of case law, statutes, and contracts; LLM summarises relevant precedents
Healthcare	Clinical decision support	Physicians query medical literature, drug databases, and patient records; RAG surfaces evidence-based recommendations
Finance	Financial research & analysis	Analysts query earnings reports, SEC filings, and market data; RAG generates comparative summaries
Customer Support	Intelligent helpdesk chatbots	Support agents query product manuals, FAQs, and ticket history; RAG reduces resolution time
Education	Personalised tutoring systems	Students ask questions; RAG retrieves relevant textbook sections and generates tailored explanations
Software Development	Code documentation Q&A	Developers query large codebases and docs; RAG answers questions with specific code references
Government	Policy and regulation search	Citizens and officials query legislation databases; RAG returns relevant policy sections with citations
HR & Internal Tools	Employee knowledge bases	Employees ask HR, IT, or policy questions; RAG retrieves from internal wikis and documents

8.2 Spotlight: Enterprise Document Q&A

This is the most common RAG deployment: building a chatbot over a company's internal documents.

Component	Typical Choice
Document Sources	Confluence, SharePoint, Google Drive, PDF libraries, internal databases
Embedding Model	OpenAI text-embedding-3-large or Cohere embed-english-v3
Vector Database	Pinecone or Azure AI Search for cloud; Chroma or Qdrant for self-hosted

LLM	GPT-4o or Claude 3.5 Sonnet (most common in enterprise)
Framework	LangChain or LlamaIndex
Evaluation	RAGAS + LangSmith for monitoring

8.3 Spotlight: RAG for Code

Code RAG helps developers interact with large, complex codebases through natural language.

- Index an entire codebase into a vector database using code-aware chunking (by function, class, or file)
- Developers ask questions like: 'Where is the authentication logic?' or 'How does the payment processing work?'
- RAG retrieves the relevant code sections and the LLM explains or modifies them
- Tools: GitHub Copilot Chat, Cursor, and custom solutions using LlamaIndex CodeSplitter

Module 9: Challenges and Best Practices

9.1 Common RAG Failure Modes

Failure Mode	Cause & Solution
Wrong chunks retrieved	Poor chunking strategy or embedding model mismatch. Solution: experiment with chunk size; use hybrid search.
Context too long for LLM	Too many chunks retrieved, exceeding context window. Solution: reduce k; use contextual compression.
Hallucination despite retrieval	LLM ignores context and generates from memory. Solution: stronger system prompt instructions; use a more instruction-following model.
Answer misses part of question	Query is multi-part; retrieval only covers one part. Solution: sub-query decomposition; multi-query retrieval.
Slow response time	Too many LLM calls, large context, or slow vector DB. Solution: cache frequent queries; use smaller re-ranker models; optimise DB queries.
Outdated information	Vector DB not updated after source documents change. Solution: implement automated re-indexing pipelines on document updates.
Poor multilingual performance	Embedding model not trained on the user's language. Solution: use multilingual embeddings (BGE-M3, Cohere multilingual).

9.2 RAG Best Practices Checklist

- **Chunking:** Test multiple chunk sizes (256, 512, 1024 tokens). Use sentence-aware or semantic chunking rather than character-based splitting.
- **Embeddings:** Use the same embedding model for indexing and querying. Changing models requires full re-indexing.
- **Retrieval:** Use hybrid search (semantic + keyword) in production. Add a re-ranker for higher precision.
- **Prompting:** Instruct the LLM explicitly to use only the provided context. Include a fallback instruction ('If the answer is not in the context, say so').
- **Evaluation:** Set up RAGAS or similar from day one. Monitor faithfulness and context relevance continuously.
- **Indexing Pipeline:** Automate re-indexing when source documents change. Track document versions.
- **Security:** Implement access control at the retrieval layer — users should only retrieve documents they are authorised to see.

9.3 RAG Security Considerations

Risk	Mitigation
Prompt Injection via Documents	Malicious text in indexed documents could manipulate the LLM. Sanitise input documents; use LLM guardrails.

Data Leakage	Users retrieving documents beyond their access level. Implement metadata-filtered retrieval with user-level access controls.
Sensitive Data in Vector DB	PII or confidential data in embeddings. Apply data governance before indexing; consider anonymisation.

KEY TAKEAWAY

RAG is not a single component — it is a pipeline. Quality must be evaluated and optimised at each stage.

The biggest gains often come from improving retrieval quality, not changing the LLM.

Start simple (naive RAG), measure quality, then add advanced techniques (hybrid search, re-ranking) where the metrics show they are needed.

Quick Revision Reference

Glossary of Key Terms

Term	Definition
RAG	Retrieval-Augmented Generation — combining a retrieval system with an LLM for grounded, accurate answers
Embedding	A numerical vector representation of text that captures its semantic meaning
Vector Database	A database optimised for storing and searching high-dimensional embedding vectors
Chunking	Splitting documents into smaller pieces for embedding and retrieval
Semantic Search	Finding similar documents based on meaning, not just keyword matching
Cosine Similarity	The most common metric for measuring similarity between two embedding vectors
Top-k Retrieval	Retrieving the k most similar chunks from the vector database
Hybrid Search	Combining vector (semantic) search with keyword (BM25) search for better recall
Re-ranking	A second model that re-scores retrieved chunks for more precise relevance
HyDE	Hypothetical Document Embedding — generate a hypothetical answer, embed it, use it for retrieval
RAGAS	RAG Assessment — a framework for automatically evaluating RAG pipeline quality
Faithfulness	Whether the generated answer is supported by the retrieved context (no hallucination)
Context Precision	The proportion of retrieved chunks that were actually relevant
Context Recall	The proportion of relevant chunks that were successfully retrieved
BM25	A classic keyword-based ranking algorithm used in hybrid search
Fine-Tuning	Training an LLM further on a specific dataset to update its weights
Knowledge Cutoff	The date after which an LLM has no knowledge from its training data
Cross-Encoder	A model that scores query-document pairs together; used for re-ranking
Metadata Filtering	Restricting retrieval to chunks that match certain metadata criteria (e.g. date, author, category)

Key Mental Models

Concept	Mental Model
RAG Formula	RAG = Retrieval System + Augmented Prompt + LLM Generation
The Open-Book Exam	RAG gives the LLM a relevant 'cheat sheet' before it answers
RAG vs Fine-tuning	RAG = update the library. Fine-tuning = update the student's brain.
Retrieval Quality Rule	Better retrieval = better answers. Fix retrieval before fixing the LLM prompt.
Chunking Trade-off	Smaller chunks = precise retrieval. Larger chunks = better context. Balance both with parent-child chunking.
Hybrid Search Rule	Always combine semantic search with keyword search in production for maximum coverage.
Evaluation First	Set up evaluation metrics before building — you cannot improve what you cannot measure.

RAG Pipeline Summary — At a Glance

Stage	Key Action	Common Tools
1. Load Documents	Ingest PDFs, web pages, databases, files	LangChain Loaders, Unstructured.io
2. Chunk Text	Split into overlapping or semantic chunks	RecursiveCharacterTextSplitter, SemanticChunker
3. Embed Chunks	Convert chunks to vectors	OpenAI, Cohere, BAAI/BGE, Sentence Transformers
4. Store Vectors	Save to vector database	Pinecone, Chroma, Qdrant, Weaviate, pgvector
5. Embed Query	Convert user question to vector	Same embedding model as step 3
6. Retrieve Chunks	Find top-k similar vectors	Vector DB similarity search + hybrid search
7. Re-rank (optional)	Re-score chunks for precision	Cohere Rerank, BGE-Reranker, ColBERT
8. Build Prompt	Combine context + user question	LangChain / LlamaIndex prompt templates
9. Generate Answer	LLM produces grounded response	GPT-4o, Claude, Llama, Gemini
10. Evaluate	Measure faithfulness, relevance, recall	RAGAS, TruLens, LangSmith

Further Learning Resources

Resource	Description
----------	-------------

LangChain RAG Tutorial	python.langchain.com — step-by-step guide to building RAG with LangChain
LlamaIndex Documentation	docs.llamaindex.ai — comprehensive RAG documentation and tutorials
RAGAS Documentation	docs.ragas.io — learn how to evaluate your RAG system
DeepLearning.AI: Building RAG	Free short course on building RAG applications (by LlamaIndex)
Pinecone Learning Centre	learn.pinecone.io — excellent tutorials on embeddings and vector search
arXiv: RAG Original Paper	'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks' — Lewis et al., 2020
Anthropic: Contextual Retrieval	Anthropic's blog post on improving RAG with contextual embeddings
Hugging Face MTEB Leaderboard	Ranks embedding models by quality — use to choose the best model for your task

AI & Data Science Study Series | RAG — Retrieval-Augmented Generation

Study. Build. Grow.